Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет <факультет>

Кафедра <кафедра>

Дисциплина: Тестирование программного обеспечения

ОТЧЁТ
по лабораторной работе
на тему

РАЗРАБОТКА ЧЕК-ЛИСТОВ И ОТЧЁТА О ТЕСТИРОВАНИИ

Студент: <И.О. Фамилия студента>

Руководитель: <И.О. Фамилия руководителя>

Минск 2026

1 ЦЕЛЬ РАБОТЫ

Освоить составление дымового и критического чек-листов, выполнить их на тестовом стенде и оформить отчёт о тестировании с метриками для веб-приложения DSLKit.

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Теория дана по книге С. С. Куликова «Тестирование программного обеспечения. Базовый курс» (EPAM Systems, 2015–2025), главы 2.2, 2.4 и раздел о метриках.

Чек-лист — набор идей для тест-кейсов. Он логичен, структурирован, полон и не избыточен; ожидаемые результаты в классическом чек-листе не пишут, он служит основой для разработки тест-кейсов и хорошо переиспользуется.

Функции приложения делят по важности на три уровня. Дымовое тестирование проверяет базовые функции, без которых приложение теряет смысл; его проводят после выхода новой сборки, чтобы решить, стоит ли продолжать, а порог прохождения обычно 100 %, и такие проверки хорошо автоматизируются. Тестирование критического пути проверяет функции, которые нужны большинству пользователей ежедневно. Расширенное тестирование охватывает остальные функции.

Отчёт о результатах тестирования фиксирует выполненные проверки, найденные дефекты и выводы. Метрики: доля выполненных тестов, доля успешных, плотность дефектов, покрытие требований; по аналогии считают покрытие классов эквивалентности и граничных условий.

3 ХОД РАБОТЫ

3.1 Вариант использования

Проверки охватывают вариант использования UC-01, связанный с требованием R4 (проверки SM-07, CP-35, CP-36); тест-кейсы к нему описаны в лабораторной работе 2.

Название.

– UC-01. Подать заявку в лист ожидания (форма «Записаться в бету» на главной странице DSLKit).

Идентификатор, версия, приоритет.

– UC-01, версия 1.0, приоритет High: единственный способ оставить контакты для получения приглашения в бету.

Краткое описание цели и задачи.

– Посетитель оставляет имя, почту, нотацию диаграмм и размер команды, чтобы команда DSLKit прислала ему приглашение, когда откроется бета. Задача формы: принять только корректную заявку, сохранить её и показать страницу благодарности.

Основной актор.

– Посетитель сайта (неавторизованный). Внешних систем и сервера в сценарии нет: заявка остаётся в браузере посетителя.

Точки входа.

– Кнопка «В бету» в шапке, кнопка «Записаться в бету» на первом экране и ссылки «Записаться в бету: UML», «Записаться в бету: BPMN» и т. п. на карточках нотаций. Все ведут к якорю #waitlist.

Предусловия.

– 1. Открыта главная страница: русская (/) или английская (/en/);

– 2. Раздел «Записаться в бету» показан, форма пуста: список «Нотация» стоит на пункте «Выберите нотацию», в поле «Размер команды» значение 0, флажок согласия снят;

– 3. Скрипт site/js/script.js загружен (иначе форма не работает).

Основной поток событий.

– 1. Посетитель нажимает «В бету» (или «Записаться в бету»); страница прокручивается к разделу #waitlist;

– 2. Посетитель вводит имя в поле «Имя» (подсказка «От 2 до 60 букв. Допустимы пробел, дефис и апостроф.»);

– 3. Посетитель вводит адрес в поле «Электронная почта» (подсказка «Адрес вида name@example.com. На него придёт приглашение.»);

– 4. Посетитель выбирает нотацию в списке: IDEF0, DFD, ERD, BPMN, UML или «Сети Петри»;

– 5. Посетитель вводит целое число в поле «Размер команды» (подсказка «Целое число от 0 до 500. Укажите 0, если работаете один.»);

– 6. Посетитель при желании вводит текст в поле «Комментарий (по

желанию)»;

– 7. Посетитель отмечает флажок «Согласен получать письма о запуске беты и с политикой конфиденциальности»;

– 8. Посетитель нажимает кнопку «Отправить»;

– 9. Система считывает шесть полей и проверяет каждое по правилам П1–П6 (функция validateWaitlist);

– 10. Система формирует запись {name, email, notation, team, comment, createdAt}, где name, email и comment очищены от пробелов по краям, team приведён к числу, createdAt — время в формате ISO 8601; добавляет запись в массив LocalStorage по ключу dslkit.waitlist и пишет в консоль «DSLKit: заявка сохранена в LocalStorage» с названием нотации;

– 11. Система открывает страницу благодарности thanks.html (для английской версии en/thanks.html) с заголовком «Спасибо, вы в списке беты».

Правила проверки полей.

– П1. Имя (#name): 2–60 символов после обрезки пробелов, первый символ буква, далее буквы, пробел, апостроф, дефис. В разметке: minlength=2, maxlength=60;

– П2. Почта (#email): вид name@domain.zone без пробелов, зона не короче двух символов, не более 120 символов после обрезки. В разметке: type=email, maxlength=120;

– П3. Нотация (#notation): idef0, dfd, erd, bpmn, uml или petri; пункт «Выберите нотацию» недопустим;

– П4. Размер команды (#team): целое от 0 до 500; допускается строка из цифр; отрицательные, дробные и нечисловые значения недопустимы. В разметке: min=0, max=500, step=1;

– П5. Комментарий (#comment): не более 500 символов (длина считается без обрезки), поле необязательное. В разметке: maxlength=500, поэтому в браузере больше 500 символов ввести нельзя;

– П6. Согласие (#consent): флажок обязательно отмечен;

– Встроенная проверка браузера отключена (form.noValidate), показываются только собственные сообщения сайта.

Альтернативные потоки.

– А1. Одно или несколько полей не прошли проверку (шаг 9): под каждым неверным полем показывается красное сообщение (role=alert, например «Имя не принято: поле пустое, короче двух символов или содержит цифры и знаки…»), поле получает aria-invalid=true, фокус переходит на первое неверное поле в порядке имя, почта, нотация, команда, комментарий, согласие. Заявка не сохраняется, страница не меняется. Поле с ошибкой перепроверяется при каждом вводе, остальные при выходе из поля. После исправления сообщение скрывается. Сценарий продолжается с шага 8;

– А2. Согласие не отмечено (шаг 9): применяется А1; сообщение «Без согласия мы не можем отправить приглашение. Поставьте галочку, чтобы продолжить.»;

– A3. Посетитель закрывает страницу или уходит со страницы до шага 8: данные не сохраняются, состояние системы не меняется;

– A4. Повторная заявка с тем же адресом (шаг 10): принимается как новая отдельная запись, сообщения о дубликате нет (дефект D-1);

– A5. Повторное нажатие «Отправить» после успешной отправки (шаг 11): игнорируется, вторая запись не создаётся (флаг submitting).

Постусловия.

– Успех: массив dslkit.waitlist в LocalStorage длиннее на одну запись, открыта страница thanks.html (en/thanks.html);

– Ошибка проверки (A1, A2): записей не добавилось, посетитель остаётся на главной странице, введённые значения остаются в полях.

Исключительные ситуации.

– E1. Запись в LocalStorage не удалась (хранилище переполнено или запрещено, шаг 10): метод storage.set возвращает false, но система всё равно открывает страницу благодарности; сообщения об ошибке нет, заявка потеряна (дефект D-2);

– E2. LocalStorage недоступен совсем (getStorage использует временное хранилище в памяти): заявка живёт только до закрытия страницы, посетитель видит страницу благодарности.

Связанные требования.

– R4 — Форма заявки и валидация;

– R1 — Доступность страниц (главная, thanks.html, privacy.html);

– R6 — Разметка и локализация (русская и английская версии);

– R9 — Локальное хранилище.

Подзадачи.

– UC-01.1 — Показ формы и подсказок к полям;

– UC-01.2 — Чтение, очистка и проверка значений по П1–П6;

– UC-01.3 — Показ ошибок, фокус и повторная проверка;

– UC-01.4 — Сохранение заявки в LocalStorage;

– UC-01.5 — Переход на страницу благодарности.

Связанные задачи и use case.

– UC-00 — Прочитать политику конфиденциальности (ссылка в тексте согласия, privacy.html);

– UC-02 — Выбрать нотацию на карточке и перейти к записи (ссылки «Записаться в бету: UML», «…: BPMN»);

– D-1 — Повторные заявки принимаются;

– D-2 — Потеря заявки при сбое хранилища.

Реализация.

– IMPL-01 — site/index.html: разметка формы #waitlist-form;

– IMPL-02 — site/js/components/waitlist-form.js: initWaitlistForm, показ ошибок, отправка;

– IMPL-03 — site/js/utils/validation.js: validateWaitlist и правила П1–П6;

– IMPL-04 — site/js/utils/storage.js: createStorage, getStorage;

– IMPL-05 — site/thanks.html и site/en/thanks.html: страница благодарности.

Тест-кейсы и проверки.

– TC-WL-001 — Все поля корректны;

– TC-WL-002…007 — Имя: границы и недопустимые символы (П1);

– TC-WL-008…011 — Почта: формат и длина (П2);

– TC-WL-012…017 — Размер команды: границы и тип (П4);

– TC-WL-018 — Нотация вне списка (П3);

– TC-WL-019, 020 — Комментарий: граница 500 (П5);

– TC-WL-021 — Согласие не дано (П6);

– TC-WL-022 — Пустая форма (A1);

– SM-07 — Форма заявки есть на главной;

– CP-35 — Валидация корректной заявки;

– CP-36 — У полей есть подписи и сообщения об ошибках.

Скриншоты интерфейса.

– SCR-01 — Главная страница, русская версия (01-home.png);

– SCR-02 — Главная страница, английская версия (02-home-en.png);

– SCR-03 — Форма заявки: ошибки при пустой отправке (03-form-errors.png);

– SCR-04 — Форма заявки: заполнена корректными данными (04-form-filled.png);

– SCR-05 — Страница благодарности (05-form-submitted.png);

– SCR-06 — Запись заявки в LocalStorage (12-uc-storage-record.png);

– SCR-07 — Сбой записи: страница благодарности без ошибки (13-uc-storage-failure.png).

Ссылки.

– Страницы проекта (при запущенном npm start):;

– http://localhost:5173/ — главная страница (русская версия);

– http://localhost:5173/#waitlist — раздел «Записаться в бету»;

– http://localhost:5173/thanks.html — страница благодарности;

– http://localhost:5173/privacy.html — политика конфиденциальности;

– http://localhost:5173/en/ — главная страница (английская версия);

– http://localhost:5173/en/thanks.html — страница благодарности (английская версия);

– Репозиторий и источники:;

– https://github.com/KsushaKhadzhinova/DSLKit-Project — репозиторий проекта;

– https://svyatoslav.biz/software_testing_book/ — книга «Тестирование программного обеспечения. Базовый курс»;

– https://github.com/KsushaKhadzhinova/report-gen — генератор отчётов.

3.2 Анализ требований и модули

Тестируется статический сайт DSLKit, запущенный локальным сервером tools/server.js. Требования и модули приведены в таблице ниже.
Таблица 3.1 – Требования и модули

ID	Модуль и требование
R1	Доступность страниц (главная, редактор, панель, настройки, английская версия)
R2	Загрузка стилей и скриптов
R3	Разбор DSL и преобразование в DOT и Mermaid
R4	Форма заявки и проверка её полей
R5	Обработка ошибок сервера: 404, 405, обход каталогов
R6	Разметка и локализация: lang, h1, title, canonical
R7	Безопасность: CSP и защитные заголовки
R8	PWA: манифест, service worker, robots, sitemap
R9	Локальное хранилище

3.3 Дымовой чек-лист

Минимальный набор из 8 проверок. Критический путь запускается только при его успешном прохождении (иначе статус Blocked).
Таблица 3.2 – Дымовой чек-лист

ID	Треб.	Проверка	Статус
SM-01	R1	Главная страница открывается (200)	Passed
SM-02	R1	Редактор открывается (200)	Passed
SM-03	R1	Панель проектов открывается (200)	Passed
SM-04	R2	Стили главной загружаются	Passed
SM-05	R2	Скрипт главной загружается	Passed
SM-06	R3	Разбор DSL возвращает 2 блока и 1 стрелку	Passed

ID	Треб.	Проверка	Статус
SM-07	R4	Форма заявки есть на главной	Passed
SM-08	R5	Несуществующий адрес даёт 404	Passed

3.4 Чек-лист критического пути

36 проверок основных сценариев (таблица ниже).

Таблица 3.3 – Чек-лист критического пути

ID	Треб.	Проверка	Статус
CP-01	R1	Страница / отдаётся как HTML	Passed
CP-02	R1	Страница /privacy.html отдаётся как HTML	Passed
CP-03	R1	Страница /thanks.html отдаётся как HTML	Passed
CP-04	R1	Страница /app/dashboard.html отдаётся как HTML	Passed
CP-05	R1	Страница /app/editor.html отдаётся как HTML	Passed
CP-06	R1	Страница /app/settings.html отдаётся как HTML	Passed
CP-07	R1	Страница /app/welcome.html отдаётся как HTML	Passed
CP-08	R1	Страница /en/ отдаётся как HTML	Passed

ID	Треб.	Проверка	Статус
CP-09	R1	Страница /en/privacy.html отдаётся как HTML	Passed
CP-10	R1	Страница /en/thanks.html отдаётся как HTML	Passed
CP-11	R6	Главная: lang=”ru англий-ская версия: lang=”en”	Passed
CP-12	R6	На каждой странице ровно один h1	Passed
CP-13	R6	У главной есть canonical и alternate	Passed
CP-14	R6	У каждой страницы есть title и meta viewport	Passed
CP-15	R2	Все локальные ссылки на ресурсы главной открываются	Passed
CP-16	R7	Нет встроенных скриптов (CSP script-src self)	Passed
CP-17	R7	Заголовок Content-Security-Policy присутствует	Passed
CP-18	R7	X-Frame-Options = DENY	Passed
CP-19	R7	X-Content-Type-Options = nosniff	Passed
CP-20	R5	POST на статику даёт 405	Passed
CP-21	R5	Выход за корень (../) не отдаёт файлы	Passed

ID	Треб.	Проверка	Статус
CP-22	R5	Страница 404 оформлена как HTML сайта	Passed
CP-23	R8	Манифест PWA: корректный JSON с иконками	Passed
CP-24	R8	Service worker доступен	Passed
CP-25	R8	robots.txt и sitemap.xml доступны	Passed
CP-26	R3	parseDsl: тип и заголовок диаграммы	Passed
CP-27	R3	parseDsl: порты блока разобраны	Passed
CP-28	R3	parseDsl: неизвестная строка попадает в skipped	Passed
CP-29	R3	parseDsl: пустой ввод не падает	Passed
CP-30	R3	toDot даёт digraph с обоими блоками	Passed
CP-31	R3	toMermaid возвращает непустой текст	Passed
CP-32	R3	sourceKey: один текст даёт один ключ, разные тексты разные	Passed
CP-33	R9	Хранилище: set/get возвращает значение	Passed
CP-34	R9	Хранилище: remove и значение по умолчанию	Passed

ID	Треб.	Проверка	Статус
CP-35	R4	Валидация заявки: корректные данные дают пустой список ошибок	Passed
CP-36	R4	Форма: у полей есть label и сообщения об ошибках	Passed

4 РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ

4.1 Общая информация

Проект DSLKit, ветка testing-course, стенд: локальный сервер Node.js, проверки выполнены автоматически командой npm run tc:lab3. Не тестировались: визуальное отображение в браузерах и работа внешнего сервиса Kroki.

4.2 Метрики

Таблица 4.1 – Метрики тестирования

Метрика	Формула	Значение
Выполнено тестов	Выполненные / Всего × 100%	100%
Успешные тесты	Passed / Выполненные × 100%	100%
Плотность дефектов	Дефекты / Количество тестов	0
Покрытие требований	Покрытые / Всего × 100%	100% (9 из 9)

Всего проверок 44: Passed 44, Failed 0, Blocked 0.

4.3 Дефекты, риски и выводы

Проваленных проверок нет, баг-репорты не требуются. На этапе отладки найдена и исправлена ошибка в самом тестовом наборе (неверный синтаксис DSL во входных данных), продукт при этом работал верно.

Риски: проверки не охватывают отрисовку в браузере и запросы к внешнему сервису диаграмм. Рекомендация: релиз возможен, браузерные сценарии проверять отдельно.

5 ВЫВОД

Составлены дымовой и критический чек-листы, выполнены на стенде, рассчитаны метрики и оформлен отчёт.